PROJECT: TECH TALENT RECRUITING WITH REGEX

datalab

As a Data Analyst at a leading global HR consultancy, your mission is to delve into an extensive database of resumes to identify suitable candidates for tech-focused roles. This task involves using regular expressions to extract key data points and applying data preprocessing techniques to organize this information effectively.

Dataset Summary

`resumes.csv`

Column	Data Type	Description
`ID`	float	Unique identifier for each resume.
`Resume_str`	object	Full text of the resume, rich with details for analysis.
`Category`	object	Job category of the resume, indicating the field of expertise.

Let's Get Started!

Embark on this analytical journey to harness advanced data analysis techniques for real-world HR challenges. This project is your chance to impact the hiring process by ensuring that tech talent finds their ideal job. Let's begin this exciting journey!

```
import pandas as pd
import re

# Load the resume dataset from a CSV file into a DataFrame
resumes = pd.read_csv('resumes.csv')
resumes.sample(3)
```

index	ID	Resume_str
1100	15119529	CONSULTANT Professional Summary Accomplished seller with marketing and entrepreneurial chops.
1138	29770086	CONSULTANT Executive Profile Marketing Executive offering outstanding presentation, communicat
1136	32637306	CONSULTANT Summary A dynamic public relations professional with 15 years of experience in devel

Rows: 3 Expand

```python
# Start coding here!
# Use as many cells as you need.
import pandas as pd
import re

# Load the resume dataset from a CSV file into a DataFrame
resumes = pd.read_csv('resumes.csv')

# Regular expression for extracting specific technical skills listed in resumes
# The word boundaries (\b) ensure that the matches are complete words, not substrings of larger words
regex_skills = r"\b(python|sql|r|excel)\b"

# regex_job_title: Captures job titles as a sequence of uppercase letters, possibly including spaces,
# periods, or hyphens to ensure it includes complete titles only.
regex_job_title = r"^([A-Z\s\.\,\-]+)\b"

# Regular expression for identifying educational degrees
# Similar to skills, it uses word boundaries to match complete words related to degrees
regex_education = r"\b(PHD|MCs|Master|BCs|Bachelor)\b"

# Lists to store the extracted information
job_titles = []
tech_skills = []
educations = []

# Loop through each resume in the DataFrame
for resume in resumes['Resume_str']:

    # Extract the job title using regex
    job_title_match = re.search(regex_job_title, resume)
    if job_title_match is not None:  # If a job title is found in the resume
        job_title = job_title_match.group(0).strip()
    else:  # If no job title is found in the resume
        job_title = ""  # Assign an empty string to indicate no job title found
    job_titles.append(job_title)  # Add the extracted job title to the list of job titles

    # Find all programming skills mentioned in the resume and make them unique
    skills_matches = re.findall(regex_skills, resume, flags=re.IGNORECASE)
    unique_skills = []
    for skill in skills_matches:  # Remove duplicates and format to title case
        skill_title = skill.title()
        if skill_title not in unique_skills:
            unique_skills.append(skill_title)
    tech_skills.append(", ".join(unique_skills))  # Convert list to comma-separated string

    # Find all educational degrees mentioned in the resume and make them unique
    education_matches = re.findall(regex_education, resume, flags=re.IGNORECASE)
    unique_education = []
    for education in education_matches:  # Remove duplicates and format to title case
        education_title = education.title()
        if education_title not in unique_education:
            unique_education.append(education_title)
    educations.append(", ".join(unique_education))  # Convert list to comma-separated string

# Add the extracted data to the DataFrame
resumes['job_title'] = job_titles
resumes['tech_skills'] = tech_skills
resumes['education'] = educations

# Filter out rows missing any job title, tech skill, or education information
resumes_filtered = resumes[(resumes['job_title'] != "") & (resumes['tech_skills'] != "") & (resumes['education'] != "")]

# Create a new DataFrame 'candidates_df' from 'resumes_filtered' with selected columns and lowercase column names for consistency
candidates_df = resumes_filtered[["ID", "job_title", "tech_skills", "education"]]
candidates_df.columns = candidates_df.columns.str.lower()

# Remove any rows that contain NaN values to ensure data integrity
candidates_df.dropna(inplace=True)

# Display the DataFrame
candidates_df.sample(10)
```

index	id	job_title	tech_skill
698	98300955	PLATFORM ARCHITECT, HEALTHCARE INCUBATION LAB	Sql
192	26503829	SENIOR TECHNICAL DESIGNER	Excel
271	18176523	SENIOR INFORMATION TECHNOLOGY MANAGER	Sql
46	34554248	HR GENERALIST	Excel
1288	25706337	DIGITAL MERCHANDISING ANALYST	Excel
1158	27726066	CONSULTANT	Excel
300	17111768	INFORMATION TECHNOLOGY PROJECT MANAGER SYSTEM ANALYSIS	Sql
106	27847081	HR SPECIALIST	Excel
771	10568183	MEDICAL SUPPORT ASSISTANT	Excel
454	16332348	VICTIM ADVOCATE	Excel

Rows: 10 Expand